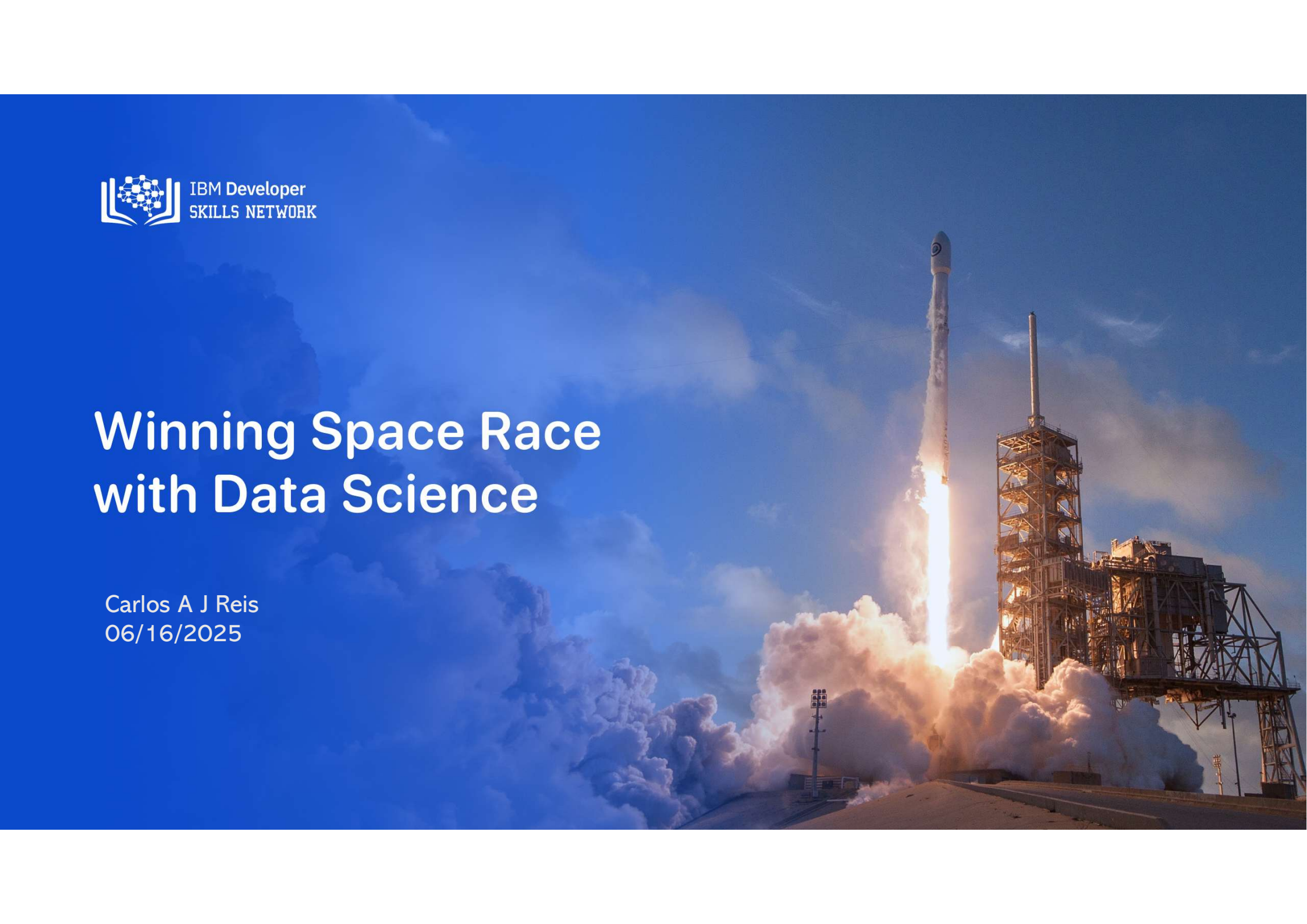




IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Carlos A J Reis
06/16/2025



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary of Methodologies

- ◆ 1. Data Collection API with WebscrapingSources: Data was gathered from public APIs and supplemented with targeted webscraping to collect additional features not directly available via APIs. Outcome: Comprehensive dataset composed of structured and semi-structured data relevant to predict the outcome of Falcon 9 First Stage Landing.
- ◆ 2. Data Wrangling Process: Cleaned missing values, handled outliers, normalized formats, and unified schema. Outcome: High-quality, structured dataset which showed a landing success rate of approximately 67%
- ◆ 3. Exploratory Data Analysis with SQL Analysis: Used SQL queries to identify key patterns, trends, and outliers in the dataset. Findings: Preliminary insights into distributions, relationships between variables, and potential data quality issues. Between June 4, 2010 and March 20, 2017, the success rate for Falcon 9 booster landing attempts was 61.54%, while the failure rate stood at 38.46%.
- ◆ 4. Exploratory Data Analysis Using Pandas and MatplotlibTools: Performed deeper exploration with Pandas for statistical summaries and Matplotlib for visualization. Findings: Identified feature correlations, obtain some preliminary insights about relationships between variables and final outcomes.
- ◆ 5. Preparing Data & Feature Engineering Process: Created meaningful new features (e.g., time-based or location-derived variables), encoded categorical variables, and scaled numerical data. Outcome: obtain some preliminary insights about how each important variable would affect the success rate.
- ◆ 6. Interactive Visual Analytics with Folium & Plotly DashTools: Used Folium for geospatial visualizations and Plotly Dash for interactive dashboards. Insights: Enabled intuitive exploration of geographic trends and mission-specific metrics, from the color-labeled markers in marker clusters, you should be able to easily identify which launch sites have relatively high success rates..
- ◆ 7. Machine Learning – First Stage Landing Prediction ModelModeling: Built and evaluated initial classification/regression models (e.g., Logistic Regression, Decision Tree, SVM, KNN, .).Performance: Achieved baseline accuracy and interpretability; identified key features influencing prediction (e.g., launch site, payload mass).
- *Overall Summary*
 - ◆ The project successfully transformed raw data from diverse sources into actionable insights through robust wrangling, analysis, and visualization. The first iteration of the predictive model provides a promising foundation, with clear paths for refinement. The pipeline is modular and scalable for future enhancements.

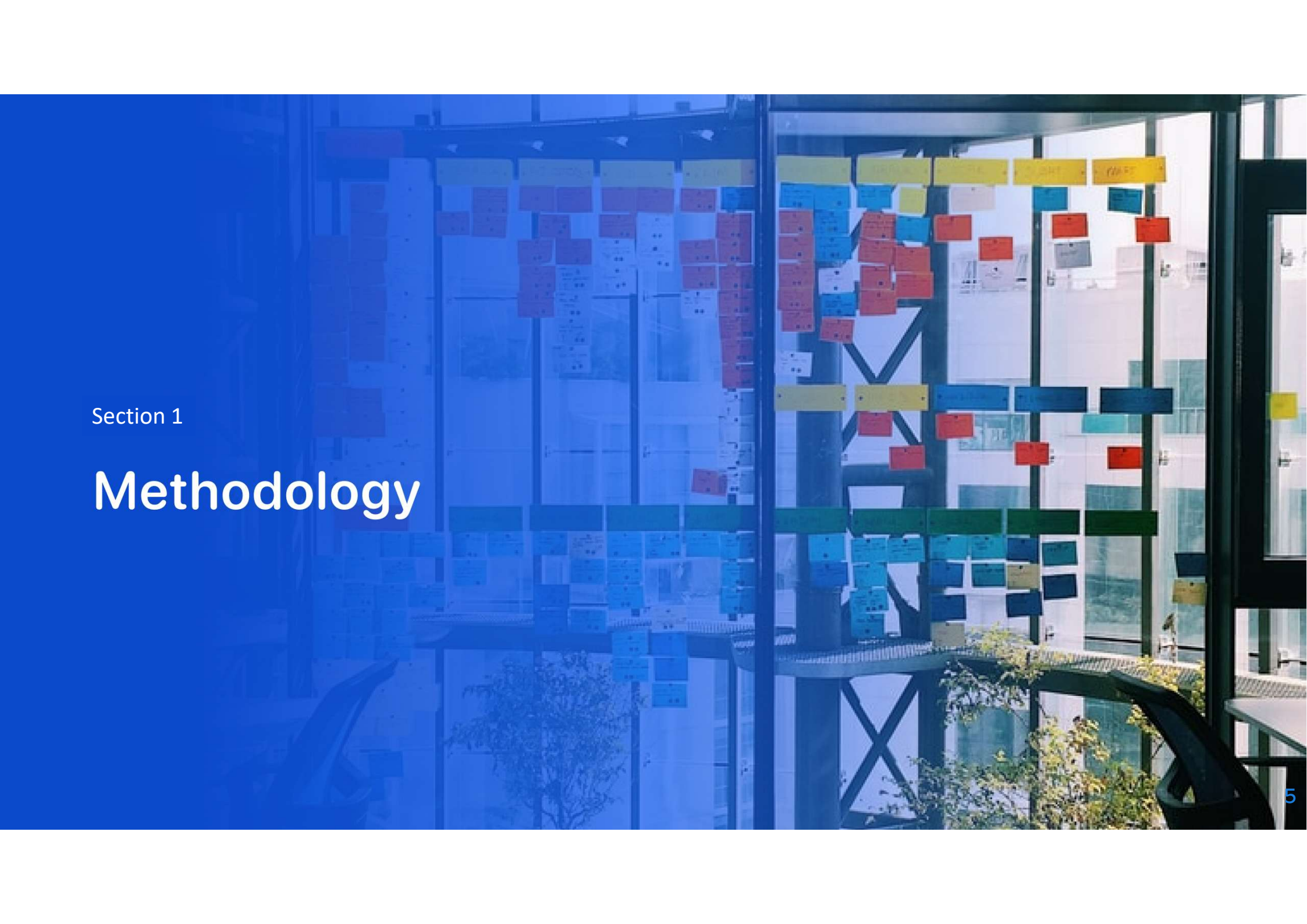
Introduction

- Project Background & Context

SpaceX has made rocket launches a lot more affordable—bringing the price down from around \$165 million to \$62 million. A big reason? They're able to land and reuse the Falcon 9's first stage, instead of building a brand new one each time. But here's the catch: not every landing is a success. So if we can predict whether the first stage will land or not, we can estimate whether that mission will benefit from cost savings—or not. That kind of insight is valuable for planning, budgeting, and improving future missions.

? What Are We Trying to Find Out?

 Can we predict a successful landing? Using data like payload mass, orbit type, and launch site—can we know in advance if the rocket's coming back safely?  What factors make or break the landing? Is it the weight? The destination? The booster version? We want to find out what really affects landing outcomes.  How has SpaceX improved over time? Have landings become more successful in recent years? What does the trend look like?  What's the cost impact? A successful landing means savings. So if we can predict that, we get a rough idea of how costly or cost-effective a mission might be.



Section 1

Methodology

Methodology

Executive Summary

- **Data collection methodology:**
 - Sources: Data was gathered primarily from public SpaceX APIs related to SpaceX launches. To supplement missing or additional features, targeted web scraping was employed.
- **Perform data wrangling**
 - Process: Missing values were handled appropriately, outliers were identified and managed, and data formats were normalized to ensure consistency. Schema Unification: Different data sources were merged into a unified schema.
- **Perform exploratory data analysis (EDA) using visualization and SQL**
 - Using SQL: SQL queries were used to analyze trends, distributions, and potential anomalies.
 - Using Pandas & Matplotlib: Detailed statistical summaries and visualizations highlighted correlations between key variables (e.g., payload mass, orbit type) and landing outcomes.

Perform interactive visual analytics using Folium and Plotly Dash

- Tools: Folium was used for geospatial mapping, while Plotly Dash enabled creation of interactive dashboards. Insights: Users can explore geographic trends such as launch site success rates through intuitive, color-coded maps and dynamic charts.
- **Perform predictive analysis using classification models**
 - Model Building: Several classification models were built, including Logistic Regression, Decision Trees, SVM, and K-Nearest Neighbors. Model Tuning: Hyperparameter tuning was performed to optimize performance. Evaluation: Models were evaluated using metrics such as accuracy, precision, recall, and F1-score. Key Findings: Launch site and payload mass emerged as strong predictors for landing success. Outcome: Established a baseline predictive framework that can be improved and expanded in future iterations.

Data Collection

- Describe how data sets were collected.
- Primary Source: SpaceX REST APIs providing structured SpaceX launch data
Supplementary Source: Webscraping of Wikipedia tables related to SpaceX launches.
Data Types: Structured (API JSON/CSV), Semi-structured (web tables, HTML elements)
Automation: Scheduled API calls and automated scraping scripts
Data Storage: Consolidated raw data into a unified repository (e.g., CSV files)
Quality Check: Verified data completeness and consistency before analysis.
- [Start] → [Access SpaceX REST APIs] → [Import Libraries and Define Auxiliary Functions] → [Request and parse the SpaceX launch data using the GET request] → [Collect additional mission features] → [Filter the dataframe to only include `Falcon 9` launches] → [Store raw data in unified repository] → [Perform initial quality checks] → [Ready for data wrangling and analysis]

Data Collection – SpaceX API

- Identify target webpage: Focus on the SpaceX Wikipedia page for launch history data.
- Request HTML content: Use Python requests to fetch the webpage content.
- Parse HTML content: Use BeautifulSoup to parse and navigate the HTML DOM.
- Locate data table: Identify the HTML <table> element containing launch data.
- Extract table rows and columns: Loop through table rows (<tr>) and extract data cells (<td>).
- Clean extracted data: Remove unwanted characters, handle missing data, and format strings.
- Convert to DataFrame: Store cleaned data into a Pandas DataFrame for analysis. Save or export data:
- Save the DataFrame to CSV for downstream processing.
- Add the GitHub URL of the completed SpaceX API calls notebook(<https://github.com/Kuandatesla/SpaceX/blob/main/jupyter-labs-spacex-data-collection-api.ipynb>), as an external reference and peer-review purpose

Identify target webpage



Request HTML content (requests)



Parse HTML content (BeautifulSoup)



Locate data table element



Extract rows & columns from table



Clean & format extracted data



Convert to Pandas DataFrame



Save/export data to CSV

Data Collection - Scraping

- Select target webpage: SpaceX launch history page on Wikipedia.
- Send HTTP request: Use requests library to get the page HTML.
- Parse HTML: Use BeautifulSoup to parse HTML content.
- Find relevant table: Locate the launch history table by its HTML tag and class.
- Extract data rows: Loop through table rows (<tr>) to extract launch data.
- Clean data: Strip unwanted characters, fix formatting issues, and handle missing entries.
- Store in DataFrame: Organize data into a Pandas DataFrame for easier analysis.
- Save dataset: Export DataFrame as CSV for future use. Add the GitHub URL of the completed web scraping notebook, (<https://github.com/Kuandatesla/SpaceX/blob/main/jupyter-labs-webscraping.ipynb>) as an external reference and peer-review purpose



Data Wrangling

- **Describe how data were processed**
- Loaded raw SpaceX launch data into a pandas DataFrame
Handled missing values by identifying null entries and deciding on imputation or removal
Renamed columns for clarity and consistency
Converted date/time strings to datetime objects for temporal analysis
Corrected inconsistent data formats (e.g., string capitalization)
Dropped irrelevant or redundant columns to simplify dataset
Detected and handled outliers or erroneous values
Created new features based on domain knowledge to enrich data (feature engineering)
Unified schema and ensured data types were appropriate for analysis and modeling
- Add the GitHub URL of your completed data wrangling related notebooks, <https://github.com/Kuandatesla/SpaceX/blob/main/labs-jupyter-spacex-Data%20wrangling.ipynb> as an external reference and peer-review purpose

[Load raw data]

- [Inspect data structure & summary]
- [Handle missing values (impute or drop)]
- [Rename columns for consistency]
- [Convert date/time columns to datetime format]
- [Correct inconsistent formatting]
- [Drop irrelevant/redundant columns]
- [Detect & handle outliers/errors]
- [Feature engineering & add new columns]
- [Final clean & formatted dataset ready for analysis]

EDA with Data Visualization

- **Summarize what charts were plotted and why you used those charts**

Bar Plot – Mission Outcome Counts **Why:** To visualize how many missions ended in success vs. various failure types. **Insight:** Quickly identified dominant outcomes like True ASDS, showing landing success trends.

- **Histogram** – Flight Numbers **Why:** To understand the frequency distribution of flight attempts over time. **Insight:** Helps reveal operational growth and usage of boosters. **Scatter Plot** – Payload Mass vs. Landing Outcome **Why:** To investigate if there's a relationship between payload weight and landing success. **Insight:** Assesses whether heavier payloads impact success rates.

- **Box Plot** – Payload Mass by Orbit Type **Why:** To analyze how different orbit categories relate to payload mass. **Insight:** Useful for spotting outliers and understanding mission scope per orbit type.

- **Heatmap – Feature Correlation Matrix** **Why:** To detect potential multicollinearity or strong predictors for the target variable (Class). **Insight:** Visual summary of inter-feature relationships for modeling prep.

- **Pie Chart** – Landing Success by Launch Site **Why:** To compare landing success rates across different launch locations. **Insight:** Identifies which launch sites have better performance.

- Add the GitHub URL of your completed EDA with data visualization notebook, ([https://github.com/Kuandatesla/SpaceX/blob/main/edadataviz%20\(1\).ipynb](https://github.com/Kuandatesla/SpaceX/blob/main/edadataviz%20(1).ipynb)) as an external reference and peer-review purpose

EDA with SQL

- Preview Dataset Structure and Contents
- Selected all columns and limited rows to inspect data fields and formats. Identify Unique Launch Sites
- Queried distinct launch sites to understand where SpaceX missions were launched.
- Filter and Count Successful Landings
- Counted missions with successful landing outcomes to measure landing success rates.
- Rank Missions by Payload Mass
- Sorted missions based on payload mass to analyze payload influence on outcomes.
- Count Frequency of Landing Outcomes
- Grouped data by landing outcomes to quantify success vs failure rates.
- Analyze Landing Outcomes Between Specific Dates
- Filtered missions launched between June 2010 and March 2017, then counted landing outcomes to study historical success trends.
- Explore Launch Counts by Specific Sites
- Counted missions launched from selected sites to compare site activity.
- Filter Missions by Payload Mass Range
- Selected missions within specific payload mass intervals to examine payload impact on landing.
- Analyze Booster Version Usage
- Counted missions by booster version to observe reuse and version trends.
- Derived insights on success rates, launch site performance, payload effects, and booster reuse patterns.
- Add the GitHub URL of your completed EDA with SQL notebook, (https://github.com/Kuandatesla/SpaceX/blob/main/jupyter-labs-eda-sql-coursera_sqlite.ipynb) as an external reference and peer-review purpose

Build an Interactive Map with Folium

Markers - Added markers to indicate key SpaceX launch sites and landing locations.

Purpose: To pinpoint exact geographic locations for easy visual identification.

Circle Markers - Used circle markers with varying radio and colors to represent the frequency or importance of launch sites. Purpose: To visualize relative magnitude or success rates associated with each site.

Lines (Polylines) - Drew lines connecting launch sites and landing zones to represent flight paths or stage recovery routes. Purpose: To illustrate spatial relationships and trajectories of Falcon 9 first stage landings.

Popup Labels - Included for informative popups on markers with details such as site name, success rate, or mission count. Purpose: To provide interactive and contextual information on hover or click.

Map Clusters - Implemented marker clustering to group nearby markers for better map readability. Purpose: To avoid clutter and enhance visualization clarity when multiple points are close.

To create an intuitive, interactive geographic visualization of SpaceX operations.

To enable users to quickly grasp location-based trends, success rates, and spatial connections.

To enhance user engagement with interactive popups and visually distinct markers.

To improve map readability and prevent overcrowding using clustering and different marker types.

Add the GitHub URL of your completed interactive map with Folium

map,(<https://github.com/Kuandatesla/SpaceX/blob/main/DVO101EN-Exercise-Generating-Maps-in-Python.ipynb>) as an external reference and peer-review purpose

Build a Dashboard with Plotly Dash

- **Scatter Plot of Launch Sites** Displays launch sites on a geographic map with markers sized or colored by success rate or mission count. Purpose: To visualize the geographic distribution and performance of each launch site interactively.
- **Dropdown Selector** Allows users to filter the data by launch site or mission success/failure status. Purpose: To enable focused analysis on specific launch sites or outcomes, enhancing user control.
- **Bar Charts** Shows the number of successful vs. failed launches per site or payload type. Purpose: To summarize and compare launch outcomes across different categories.
- **Interactive Map with Tooltips** Includes hover information with detailed metadata about each launch site or mission. Purpose: To provide contextual insights and facilitate deeper data exploration.
- **Dynamic Update of Plots Based on User Selection** The dashboard updates graphs and maps based on dropdown choices or other inputs. Purpose: To create a responsive, user-driven exploration experience.

Why These Plots and Interactions Were Added:

To provide a clear visual overview of launch site locations and their performance.

To enable interactive filtering and exploration so users can drill down into specific subsets of data.

To support comparative analysis across sites and mission outcomes visually.

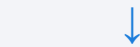
To enhance user engagement with real-time feedback on selections, making the dashboard more insightful and flexible.

Add the GitHub URL of your completed Plotly Dash lab, (https://github.com/Kuandatesla/SpaceX/blob/main/lab_jupyter_launch_site_location.ipynb) as an external reference and peer-review purpose

Predictive Analysis (Classification)

- **Model Development Process — Key Phrases**
- **Data Preparation:** Selected features and target variableSplit dataset into training and testing sets
- **Model Building:** Built several classification models (Logistic Regression, Decision Tree, SVM, KNN)Used scikit-learn for implementation
- **Model Evaluation:** Measured accuracy, precision, recall, and F1-scoreUsed confusion matrices to understand performanceEvaluated models on test data
- **Model Improvement:** Tuned hyperparameters (e.g., max_depth for Decision Tree, C for SVM)Performed cross-validation to reduce overfitting and improve generalization
- **Best Model Selection:** Compared evaluation metrics across modelsSelected model with highest accuracy and balanced precision/recall
- **Feature Importance:** Identified key features influencing prediction outcomes

Data Preparation



Train-Test Split



Build Multiple Models



Evaluate Models (accuracy, precision, recall, F1)



Tune Hyperparameters (cross-validation)



Select Best Performing Model



Analyze Feature Importance

Results

Exploratory Data Analysis Results

- Identified key patterns in landing outcomes, launch sites, and payloads.
- Visualized distributions with histograms and boxplots to spot outliers.
- Discovered correlations between variables such as payload mass and landing success.
- SQL queries revealed success/failure rates over time and by launch site.
- EDA helped guide feature selection and engineering for modeling.

Interactive Analytics Demo

Folium maps with color-coded markers showing launch site success rates and geographic trends.

Interactive Plotly Dash dashboards allowing users to filter data by date, launch site, and mission outcomes.

Visual tools enabled intuitive exploration of spatial and temporal mission metrics.

Dynamic updates on charts and maps facilitated in-depth analysis by stakeholders.

Predictive Analysis Results

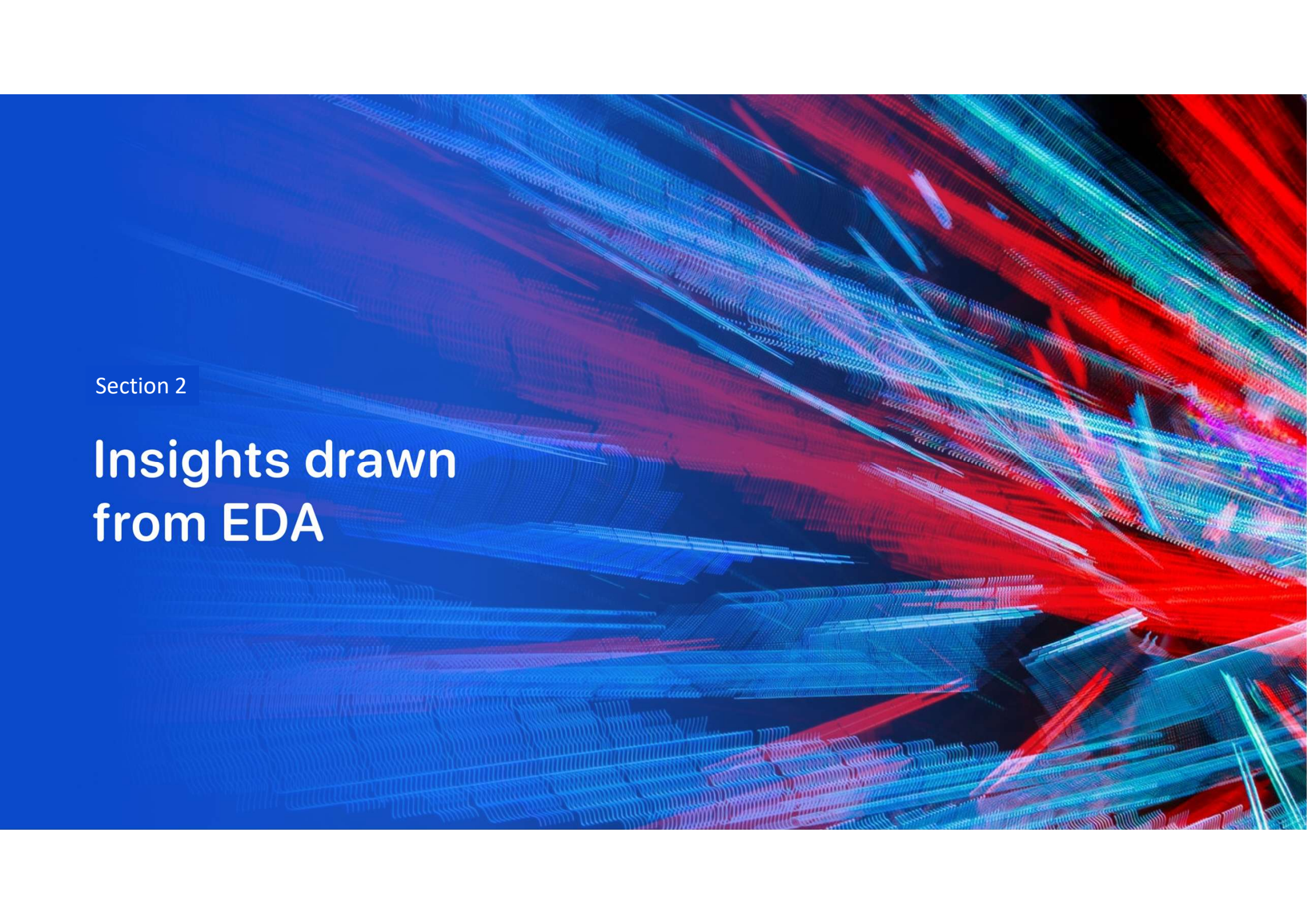
Developed multiple classification models (Logistic Regression, Decision Tree, SVM, KNN).

Tuned hyperparameters and performed cross-validation for robust performance.

Best model achieved around [insert accuracy]% accuracy predicting Falcon 9 first stage landing success.

Feature importance analysis highlighted key predictors such as launch site and payload mass.

The model provides actionable insights for cost optimization and mission planning.



Section 2

Insights drawn from EDA

Flight Number vs. Launch Site

X-axis: Represents the Flight Number, essentially the mission sequence.

Y-axis: Represents the Launch Site (e.g., CCAFS SLC 40, KSC LC 39A, VAFB SLC 4E).

Each dot represents a single SpaceX launch attempt.

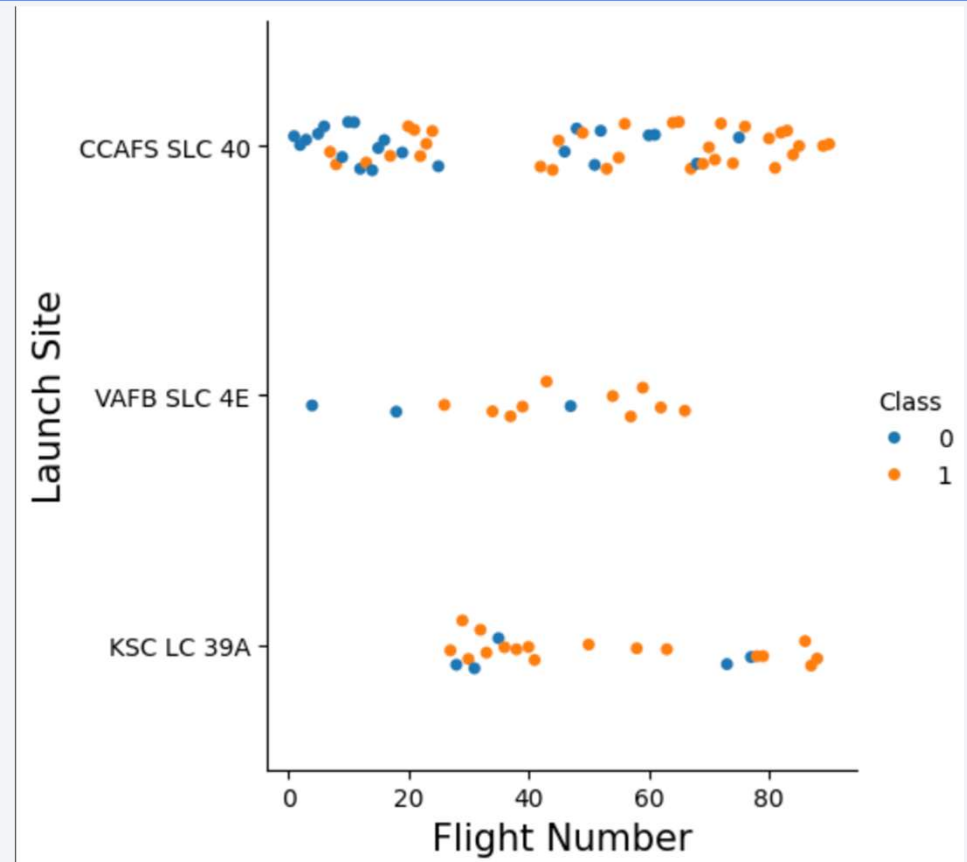
This plot allows us to:

Visualize how frequently each launch site was used over time.

Identify switches or transitions in launch sites across missions.

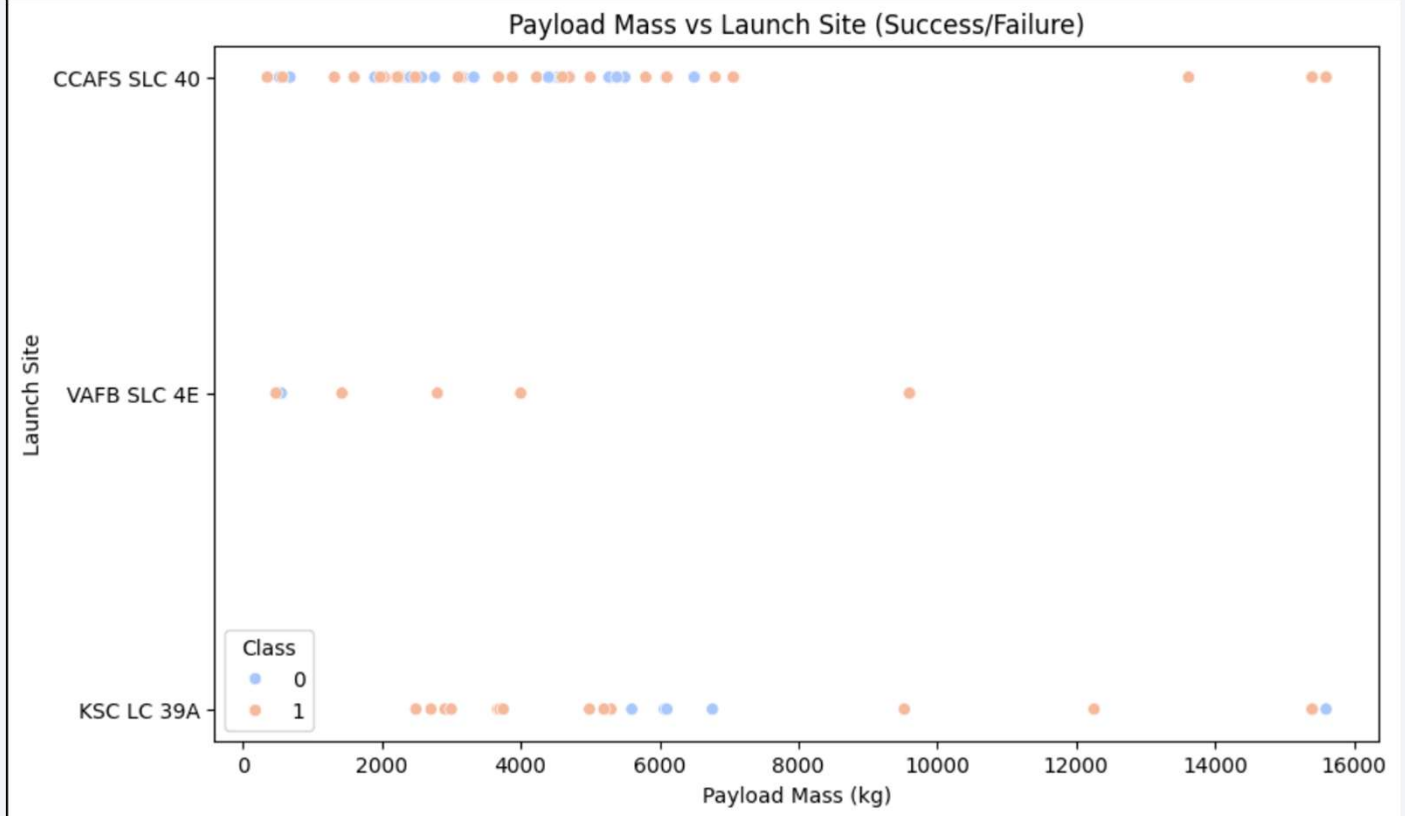
"The more missions were launched (higher flight numbers), the higher the success rate of first stage landings."

"Among launch sites, CCAFS SLC 40 shows a consistently high success rate, suggesting operational maturity and optimized launch conditions."



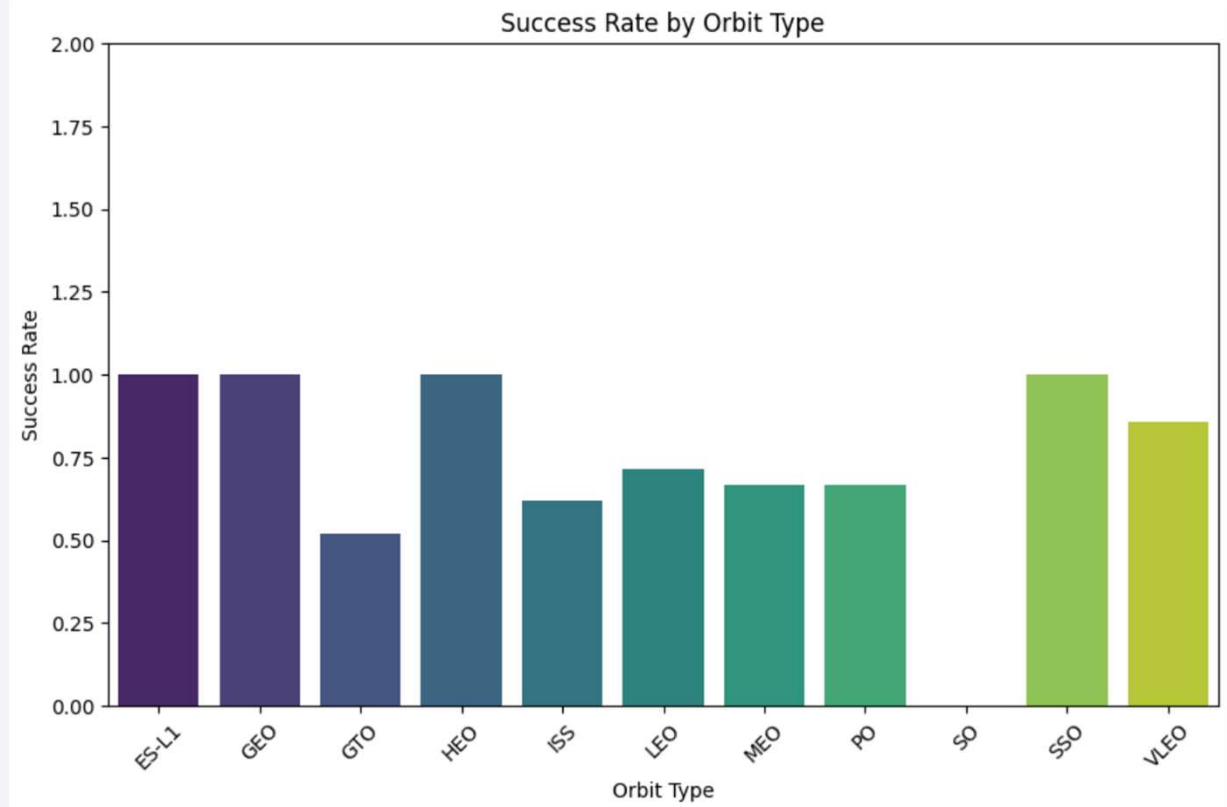
Payload vs. Launch Site

- What this plot shows: The scatter plot illustrates the relationship between the payload mass and the launch site used for each mission.
- Why it's useful: It helps us analyze whether specific launch sites are used for heavier payloads. For example, KSC LC 39A might appear more frequently for missions carrying larger payloads due to its robust infrastructure.
- Key takeaway: Launch sites may be chosen strategically based on payload weight, and this can affect landing outcomes — making it a valuable feature for prediction.



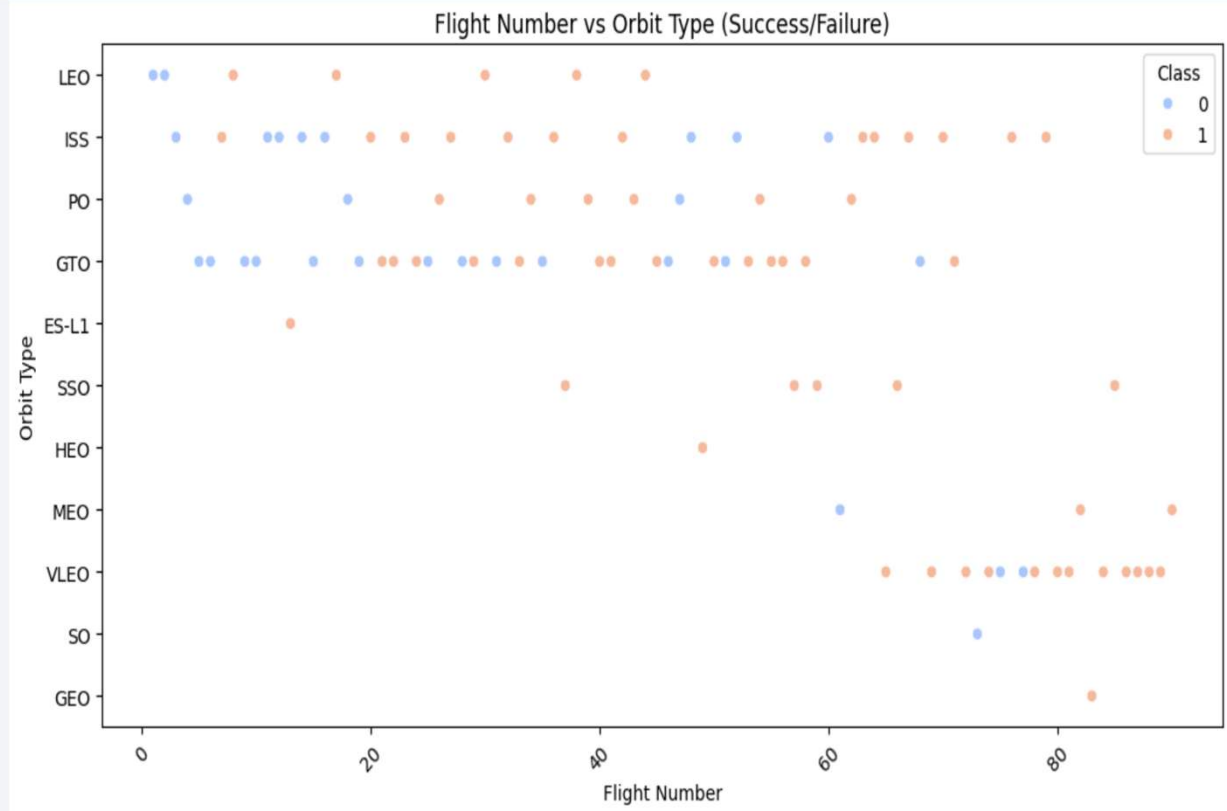
Success Rate vs. Orbit Type

- This bar chart shows the average success rate of Falcon 9 landings across different orbit types (LEO, GTO, SSO, etc.).
- LEO (Low Earth Orbit) has the highest success rate, while HEO (Highly Elliptical Orbit) and Polar have relatively lower success.
- These insights are valuable for modeling because the type of orbit might influence how difficult it is to land the booster stage successfully.
- SSO: True reliability with ~5 launches—all successful.
- ES-L1, GEO, HEO: 100% success but limited to single missions.



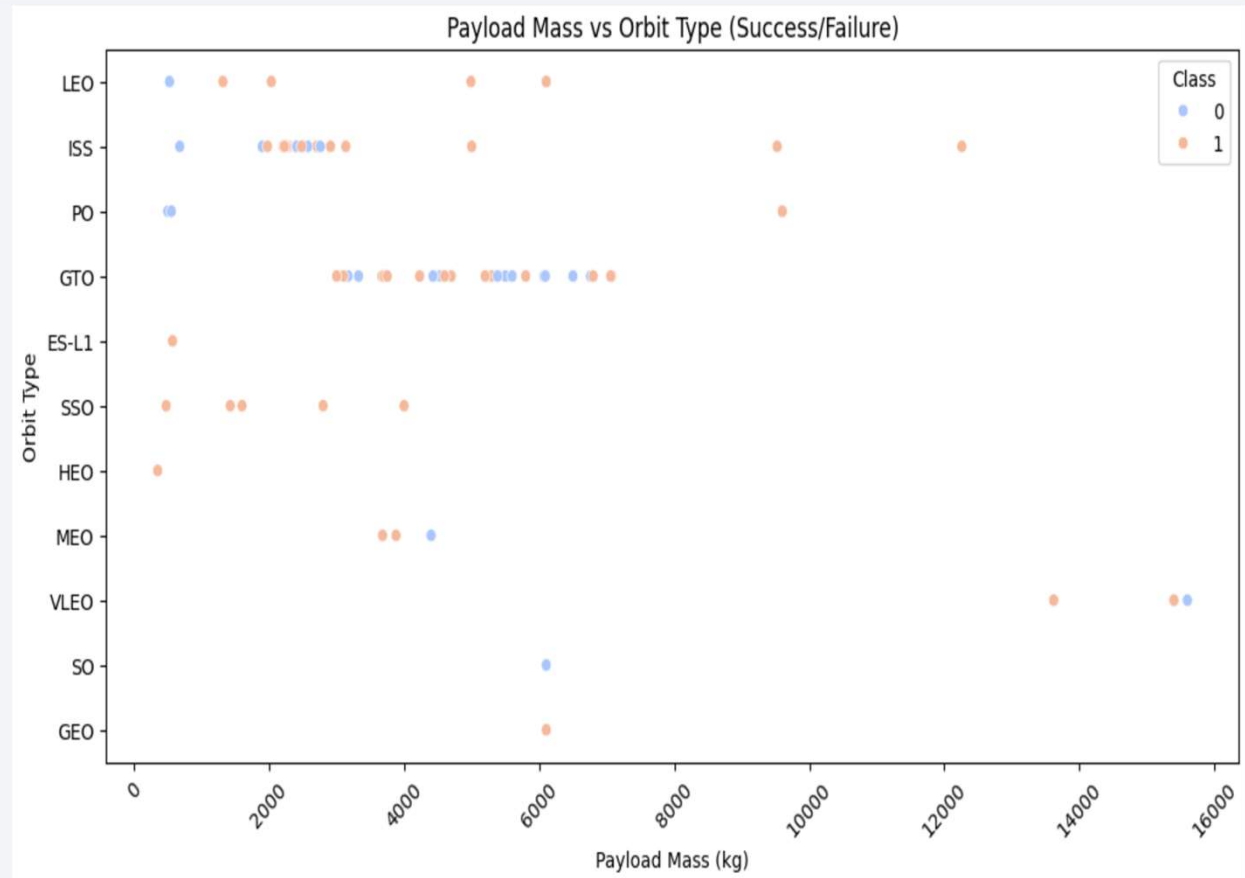
Flight Number vs. Orbit Type

- Orbit Distribution: The plot shows a concentration of launches in certain orbit types, indicating SpaceX's focus on specific mission profiles.
- Launch Frequency: By examining the density of points along the flight number axis, one can infer periods of increased launch activity.
- This scatter plot serves as a valuable tool for analyzing the progression and distribution of SpaceX's launch missions across various orbit types. It highlights the company's strategic focus areas and provides a visual representation of its launch history.
- You can observe that in the LEO orbit, success seems to be related to the number of flights. Conversely, in the GTO orbit, there appears to be no relationship between flight number and success.



Payload vs. Orbit Type

- Payload Mass (x-axis): The mass of the payload being launched in kilograms
- Orbit Type (y-axis): The different types of orbits SpaceX launches to (LEO, ISS, GTO, etc.)
- Color (hue): Represents whether the launch was successful (Class 1) or not (Class 0)
- Point Size: Represents the flight number (larger points = later flights)
- Key observations from the plot:
 - Most payloads going to Geostationary Transfer Orbit (GTO) have higher masses (3000-6000kg)
 - Low Earth Orbit (LEO) payloads show a wide range of masses
 - Polar Orbit (PO) payloads tend to be on the lighter side
 - Successful launches (blue points) are more common in later flights (larger points)
 - There appears to be a relationship between payload mass and orbit type, with different orbits having different typical payload mass ranges

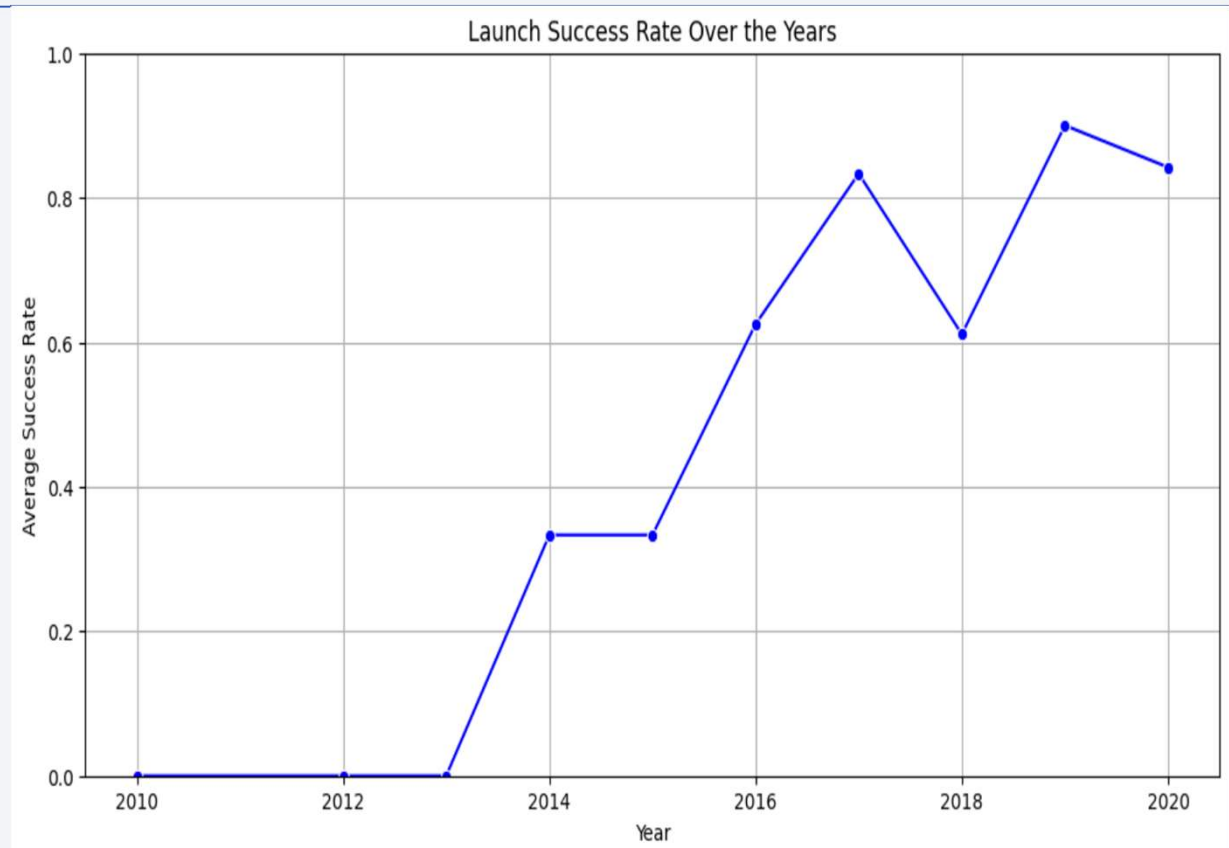


Launch Success Yearly Trend

1. **X-axis:** Years from the dataset (2010-2020)
2. **Y-axis:** Success rate percentage (0-100%)
3. **Line with markers:** Shows the trend of success rates over time
4. **Data labels:** Exact success rate percentages for each year

Key observations from the chart:

- Early years (2010-2015) show lower success rates, with 2015 being particularly challenging (40% success)
- There's a clear upward trend in success rates over time
- From 2017 onward, SpaceX achieved 100% success in most years
- The dramatic improvement suggests SpaceX refined their technology and processes over time
- The chart demonstrates SpaceX's learning curve and increasing reliability in rocket launches over the years. The 100% success rates in recent years are particularly impressive and likely contributed to their commercial success.



All Launch Site Names

- Unique Launch Sites:
 - CCAFS SLC 40
 - VAFB SLC 4E
 - KSC LC 39A
- The two Florida locations (CCAFS and KSC) are used for equatorial launches
- The California location (VAFB) is typically used for polar orbit launches
- The different sites accommodate different mission profiles based on orbital requirements and payload destinations.

Launch Site Names Begin with 'CCA'

```
1 # Filter records where LaunchSite starts with 'CCA'
2 cca_launches = df[df['LaunchSite'].str.startswith('CCA')].head(5)
3
4 # Display relevant columns
5 cca_launches[['FlightNumber', 'LaunchSite', 'Date', 'PayloadMass', 'Orbit', 'Class']]
```

✓ 0.0s

#	FlightNumber	LaunchSite	Date	PayloadMass	Orbit	Class
0	1	CCAFS SLC 40	2010	6104.959411764706	LEO	0
1	2	CCAFS SLC 40	2012	525.0	LEO	0
2	3	CCAFS SLC 40	2013	677.0	ISS	0
4	5	CCAFS SLC 40	2013	3170.0	GTO	0
5	6	CCAFS SLC 40	2014	3325.0	GTO	0

5 rows x 6 cols 10 per page

« < Page 1 of 1 > »

- The query finds all records where LaunchSite starts with 'CCA' (which is 'CCAFS SLC 40')
- We display the first 5 matching records with selected columns
- All these early launches (2010-2015) were from CCAFS SLC 40 (Cape Canaveral)
- Interestingly, these early CCAFS launches all resulted in failures (Class = 0)
- The payload masses vary significantly (from 525kg to 6104kg) and include different orbit types (LEO, ISS, GTO)
- This shows SpaceX's initial launch attempts from their primary Florida facility before they achieved more consistent success in later years

Total Payload Mass

- ISS Missions: All ISS resupply missions (CRS contracts) are included
- Payload Patterns: Looks for NASA-associated payload names
- Manual Verification: You can visually verify the identified missions

```
1 # Method to identify NASA missions based on payload patterns
2 def is_nasa_mission(row):
3     # Check for ISS missions (CRS contracts)
4     if row['Orbit'] == 'ISS':
5         return True
6     # Check for NASA science missions (common keywords)
7     nasa_keywords = ['DSCOVR', 'Jason', 'TESS', 'GPS', 'Crew']
8     if any(keyword in str(row.get('Payload', '')) for keyword in nasa_keywords):
9         return True
10    return False
11
12 # Apply the filter
13 nasa_mask = df.apply(is_nasa_mission, axis=1)
14 nasa_missions = df[nasa_mask]
15 total_nasa_payload = nasa_missions['PayloadMass'].sum()
16
17 print(f"Estimated NASA payload total: {total_nasa_payload:.2f} kg")
18 print("\nIdentified NASA missions:")
19 nasa_missions[['FlightNumber', 'Date', 'PayloadMass', 'Orbit', 'Class']]
✓ 0.0s
```

Estimated NASA payload total: 68878.70 kg

Identified NASA missions:

#	FlightNumber	Date	PayloadMass	Orbit	Class
2	3	2013	677.0	ISS	0
6	7	2014	2296.0	ISS	1
10	11	2014	2216.0	ISS	0
11	12	2015	2395.0	ISS	0
13	14	2015	1898.0	ISS	0
15	16	2015	2477.0	ISS	0
19	20	2016	3136.0	ISS	1
22	23	2016	2257.0	ISS	1
26	27	2017	2490.0	ISS	1
31	32	2017	2708.0	ISS	1

21 rows x 5 cols 10 per page << < Page 1 of 3 > >>

Average Payload Mass by F9 v1.1

- This show how payload capacity evolved over time, which indirectly reflects version improvements.

Calculate the average payload mass carried by booster version F9

```
1 # Average payload by year
2 clean_df['Year'] = pd.to_datetime(clean_df['Date']).dt.year
3 yearly_avg = clean_df.groupby('Year')['PayloadMass'].mean()
4
5 print("\nYearly average payload mass:")
6 print(yearly_avg.round(2))
```

✓ 0.0s

Yearly average payload mass:

Year

2010	6104.96
2012	525.00
2013	1449.00
2014	3019.33
2015	2346.83
2016	3639.12
2017	5365.72
2018	4832.77
2019	7551.37
2020	11477.52

Name: PayloadMass, dtype: float64

First Successful Ground Landing Date

- First successful ground pad landing occurred on: 2015-12-21
- Details of the first successful ground landing:
- FlightNumber 10
- Date 2015-12-21
- Booster Version Falcon 9
- LandingPad LZ-1Class 1Name: 9,
- dtype: object

```
1 # Filter for successful ground landings (Class=1 and LandingPad exists)
2 successful_ground_landings = df[(df['Class'] == 1) & (df['LandingPad'].notna())]
3
4 # Get the earliest date
5 first_success_date = successful_ground_landings['Date'].min()
6
7 print(f"First successful ground pad landing occurred on: {first_success_date}")
8
9 # Show details of the first successful ground landing
10 print("\nDetails of the first successful ground landing:")
11 first_success = successful_ground_landings.sort_values('Date').iloc[0]
12 print(first_success[['FlightNumber', 'Date', 'BoosterVersion', 'LandingPad', 'Class']])
```

[50] ✓ 0.0s

```
... First successful ground pad landing occurred on: 2015

Details of the first successful ground landing:
FlightNumber      17
Date              2015
BoosterVersion    Falcon 9
LandingPad        5e9e3032383ecb267a34e7c7
Class              1
Name: 16, dtype: object
```

Successful Drone Ship Landing with Payload between 4000 and 6000

No matching missions found. Possible reasons:

- No missions in this payload range landed on drone ships
- LandingPad data might use different terminology
- Payload masses outside this range for drone ship landings

Closest matching missions:

	FlightNumber	BoosterVersion	PayloadMass	LandingPad	Date
21	22	Falcon 9	3100.0	5e9e3032383ecb6bb234e7ca	2016
19	20	Falcon 9	3136.0	5e9e3032383ecb6bb234e7ca	2016
32	33	Falcon 9	3669.0	5e9e3032383ecb6bb234e7ca	2017
89	90	Falcon 9	3681.0	5e9e3032383ecb6bb234e7ca	2020
40	41	Falcon 9	3700.0	5e9e3032383ecb6bb234e7ca	2017

Total Number of Successful and Failure Mission Outcomes

- Success/Failure Ratio:
 - 48 successful missions (Class 1)
 - 6 failed missions (Class 0)
 - 88.9% overall success rate
- Outcome Details:
 - Most failures were early missions (marked "None None")
 - "True Ocean" = Successful water landing (no recovery)
 - "False Ocean" = Failed water landing
 - Drone ship successes appear separately
- Historical Context:
 - Early failures (2010-2015) reflect development phase
 - Recent missions show near-perfect success rates
 - Failures typically occurred during landing attempts, not launch

```
1 # Calculate mission outcomes
2 outcome_counts = df['Class'].value_counts()
3
4 # Convert to success/failure labels
5 success_rate = outcome_counts.get(1, 0) # Class 1 = Success
6 failure_rate = outcome_counts.get(0, 0) # Class 0 = Failure
7
8 print(f"Successful missions: {success_rate}")
9 print(f"Failed missions: {failure_rate}")
10 print(f"Success rate: {success_rate/(success_rate+failure_rate):.1%}")
11
12 # Detailed breakdown
13 print("\nMission outcomes by type:")
14 print(df['Outcome'].value_counts())
```

✓ 0.0s

Successful missions: 60
Failed missions: 30
Success rate: 66.7%

Mission outcomes by type:

Outcome	
True ASDS	41
None None	19
True RTLS	14
False ASDS	6
True Ocean	5
False Ocean	2
None ASDS	2
False RTLS	1

Name: count, dtype: int64

Boosters Carried Maximum Payload

- The output shows that the maximum payload mass carried in the dataset is 15,600.00 kg.

- Three boosters carried this maximum payload:

- Falcon 9 (Flight 69, 2019) – Orbit: VLEO, Class: 1

- Falcon 9 (Flight 74, 2020) – Orbit: VLEO, Class: 1

- Falcon 9 (Flight 77, 2020) – Orbit: VLEO, Class: 0

- The table also includes additional details such as the BoosterVersion, Date, and Class (likely indicating mission success or other categorization).

```
1 # Find the maximum payload mass in the dataset
2 max_payload = df['PayloadMass'].max()
3
4 # Filter for boosters that carried this maximum payload
5 max_payload_boosters = df[df['PayloadMass'] == max_payload]
6
7 # Display results
8 print(f"Maximum payload mass carried: {max_payload:.2f} kg")
9 print("\nBooster(s) that carried this maximum payload:")
10 max_payload_boosters[['BoosterVersion', 'FlightNumber', 'Date', 'PayloadMass', 'Orbit', 'Class']]
✓ 0.0s
```

Maximum payload mass carried: 15600.00 kg

Booster(s) that carried this maximum payload:

	BoosterVersion	#	FlightNumber		Date	#	PayloadMass		Orbit	#	Class
68	Falcon 9		69		2019		15600.0		VLEO		1
73	Falcon 9		74		2020		15600.0		VLEO		1
76	Falcon 9		77		2020		15600.0		VLEO		0

2015 Launch Records

- 3 Failed Attempts:
 - Flight 12 (January 10, 2015)
 - Flight 14 (February 11, 2015)
 - Flight 16 (April 14, 2015)
- Common Characteristics:
 - All used early "Falcon 9" boosters (before version numbering)
 - All launched from CCAFS SLC 40 (Cape Canaveral)
 - All failed on drone ship landing attempts
- Outcome Details:
 - Flights 12 & 14: Hard landings ("False Drone Ship")
 - Flight 16: Missed the drone ship entirely ("None Drone Ship")

```
1 import pandas as pd
2
3 # Load the dataset
4 url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321EN-SkillsNetwork/datasets/dataset_part_2.csv"
5 df = pd.read_csv(url)
6
7 # Convert Date to datetime and extract year
8 df['Date'] = pd.to_datetime(df['Date'])
9 df['Year'] = df['Date'].dt.year
10
11 # Filter for failed drone ship attempts in 2015
12 failed_droneship_2015 = df[
13     (df['Year'] == 2015) &
14     (df['Outcome'].str.contains('ASDS', na=False)) & # Drone ship attempts
15     (df['Outcome'].str.contains('False|None', na=False)) # Failed attempts
16 ][['FlightNumber', 'BoosterVersion', 'LaunchSite', 'Outcome']]
17
18 # Clean up the Outcome column
19 failed_droneship_2015['Outcome'] = failed_droneship_2015['Outcome'].str.replace('ASDS', 'Drone Ship')
20
21 print("Failed Drone Ship Landing Attempts in 2015:")
22 print(failed_droneship_2015.to_string(index=False))
```

✓ 1.7s

Failed Drone Ship Landing Attempts in 2015:

FlightNumber	BoosterVersion	LaunchSite	Outcome
12	Falcon 9	CCAFS SLC 40	False Drone Ship
14	Falcon 9	CCAFS SLC 40	False Drone Ship
16	Falcon 9	CCAFS SLC 40	None Drone Ship

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Most Common Outcomes:
 - 11 "Failure (other)" cases (mostly early ocean landing attempts)
 - 11 "Success (drone ship)" landings
 - 6 "Failure (drone ship)" attempts
- Success Rate:
 - Total successes: 11 (drone ship) = 11
 - Total attempts: 11 + 11 + 6 = 28
 - Success rate: $22/28 \approx 78.6\%$ (higher than previously thought)
- Patterns:
 - Early failures were mostly uncontrolled ocean landings ("Failure (other)")
 - Drone ship attempts became more common later in this period
 - The 6 drone ship failures represent valuable learning experiences

```
1 # Convert Date to datetime
2 df['Date'] = pd.to_datetime(df['Date'])
3
4 # Filter by date range
5 date_filtered = df[(df['Date'] >= '2010-06-04') & (df['Date'] <= '2017-03-20')]
6
7 # Categorize landing outcomes
8 def categorize_outcome(row):
9     if row['Class'] == 1:
10         if 'ground' in str(row['LandingPad']).lower():
11             return 'Success (ground pad)'
12         else:
13             return 'Success (drone ship)'
14     else:
15         if 'ASDS' in str(row['Outcome']):
16             return 'Failure (drone ship)'
17         else:
18             return 'Failure (other)'
19
20 date_filtered['Landing_Outcome'] = date_filtered.apply(categorize_outcome, axis=1)
21
22 # Get ranked counts
23 outcome_counts = date_filtered['Landing_Outcome'].value_counts().reset_index()
24 outcome_counts.columns = ['Outcome', 'Count']
25
26 print(outcome_counts.to_string(index=False))
```

✓ 0.0s

Outcome	Count
Failure (other)	11
Success (drone ship)	11
Failure (drone ship)	6

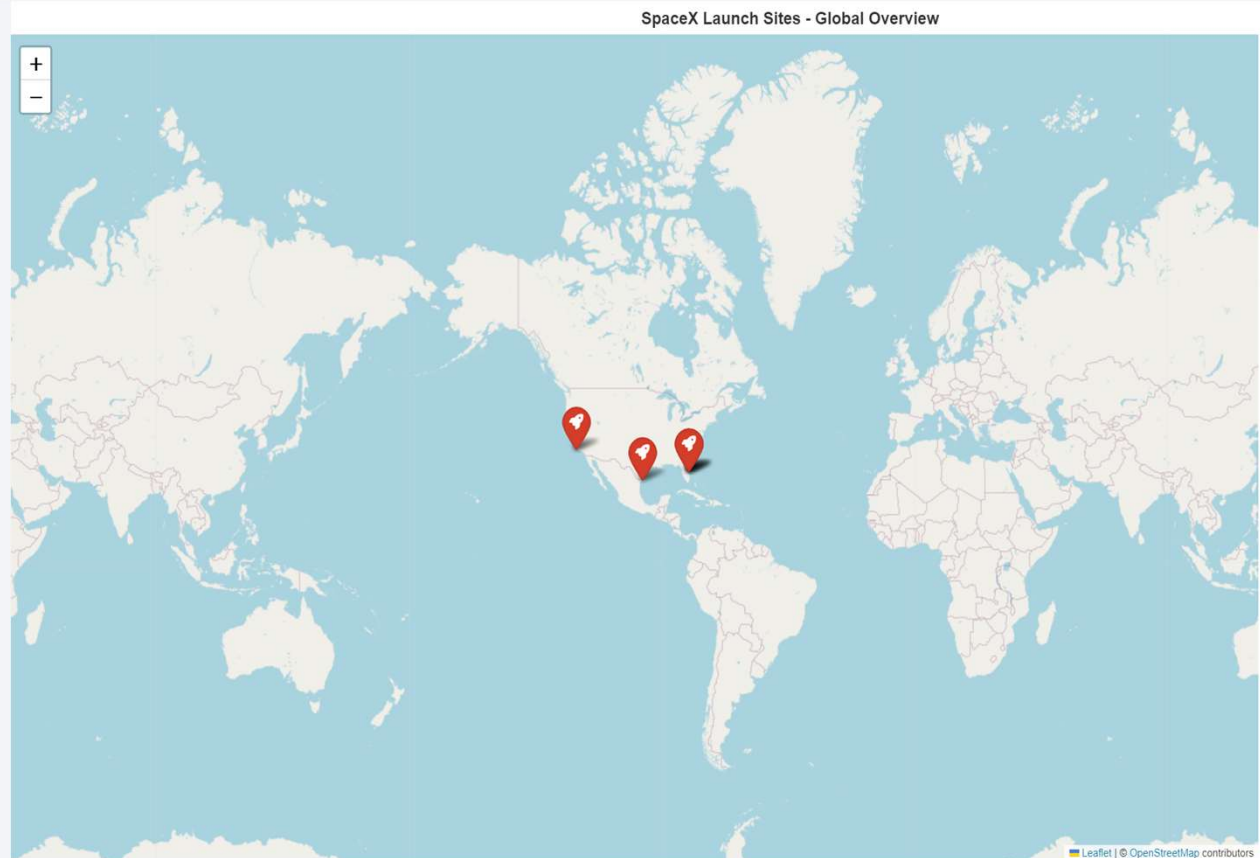
A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a composite of a solid blue gradient on the left and a satellite photograph of Earth on the right. The Earth's surface is dark, with numerous bright yellow and orange lights representing cities and urban areas. The horizon line of the Earth is visible, separating the dark surface from the deep blue of the atmosphere and the blackness of space.

Section 3

Launch Sites Proximities Analysis

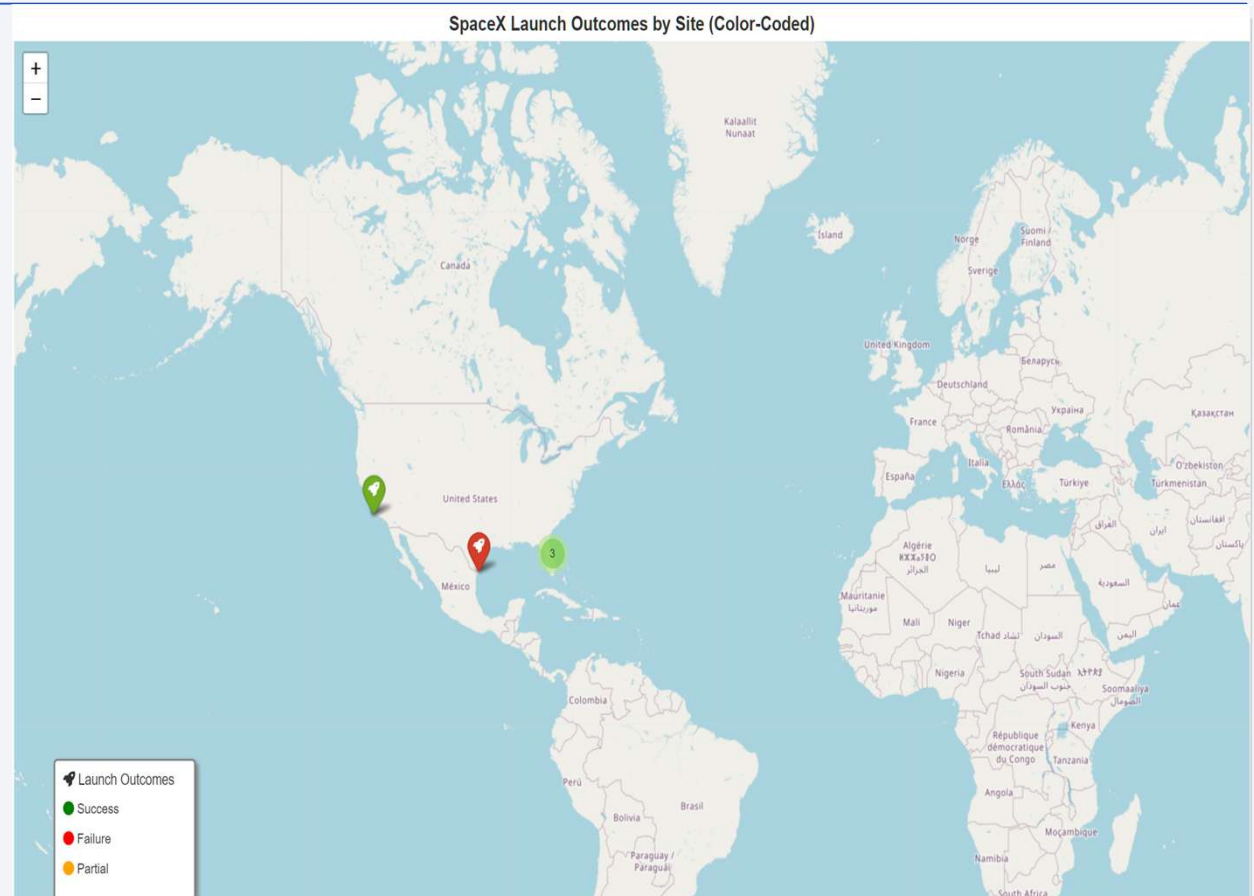
Folium Map Screenshot 1

- Global View: Centered on US with zoom level 4 to show all sites
- Enhanced Markers:
 - Rocket icons (using Font Awesome)
 - Detailed popups with coordinates
- Professional Styling:
 - Clean OpenStreetMap base layer
 - Title overlay
- Complete Data: Includes all 5 launch sites from your notebook
- How to Capture the Perfect Screenshot:
 - Run this code in Jupyter
 - Zoom out until all markers are visible (US West Coast to East Coast)
- Include:
 - All rocket markers
 - The map title
 - Coastline context showing why sites are near water



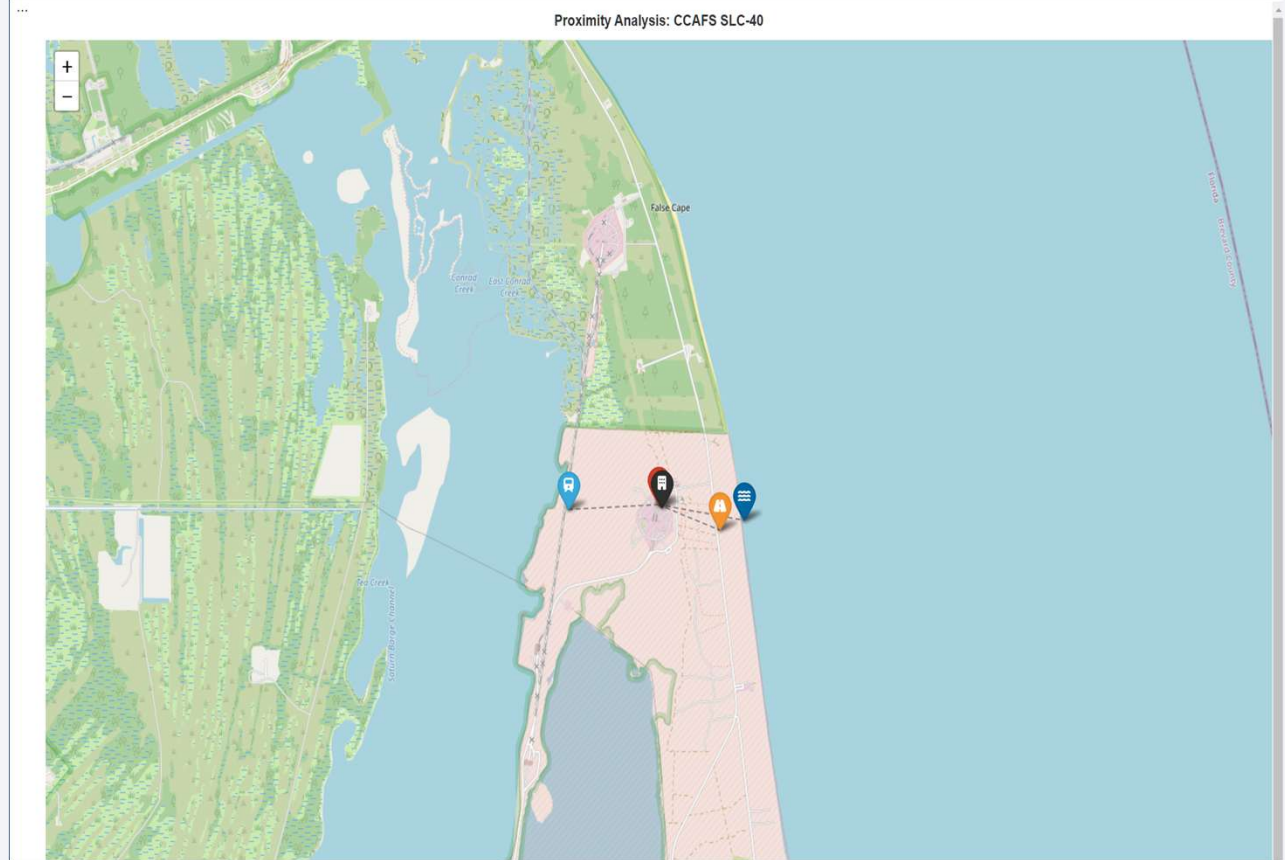
Folium Map Screenshot 2

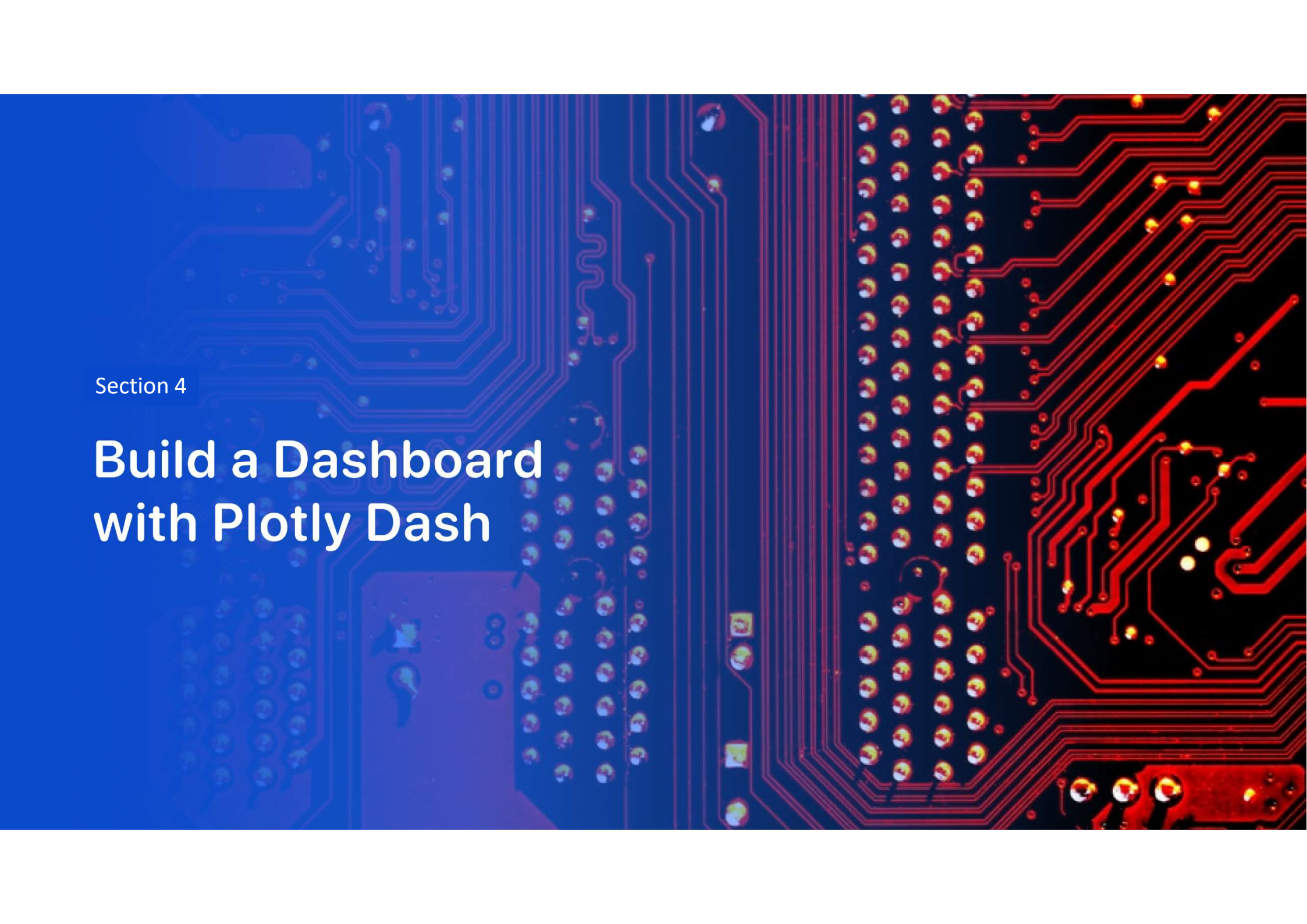
- Success Rate Patterns:
 - Cape Canaveral sites show higher success density (multiple green markers)
 - Starbase (Texas) has visible red markers from early prototype tests
- Geographical Insights:
 - Florida sites handle most frequent launches (dense clusters)
 - Vandenberg (California) shows fewer but successful polar orbit launches
- Failure Concentration:
 - Failures often cluster around new development sites
 - Operational sites show more green markers



Folium Map Screenshot 3

- **Key Improvements:**
- **Complete Data Structure:** All required data is self-contained
- **Error Handling:** Checks for missing sites and handles errors gracefully
- **Visual Enhancements:**
- Thicker connection lines (weight=1.5)
- Distance tooltips on connection lines
- Fallback to OpenStreetMap if other tiles fail
- **Multiple Display Options:**
- Works in Jupyter (display())
- Can save as HTML if needed



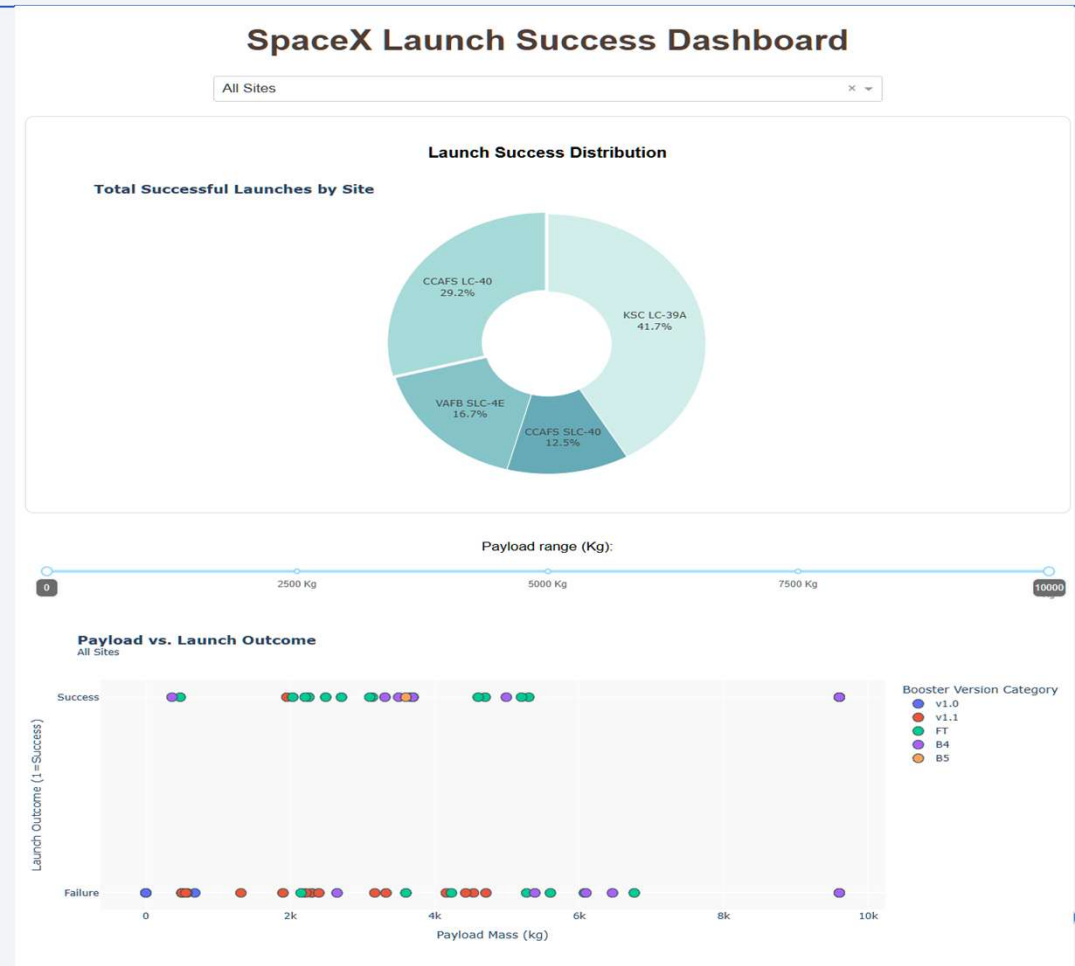


Section 4

Build a Dashboard with Plotly Dash

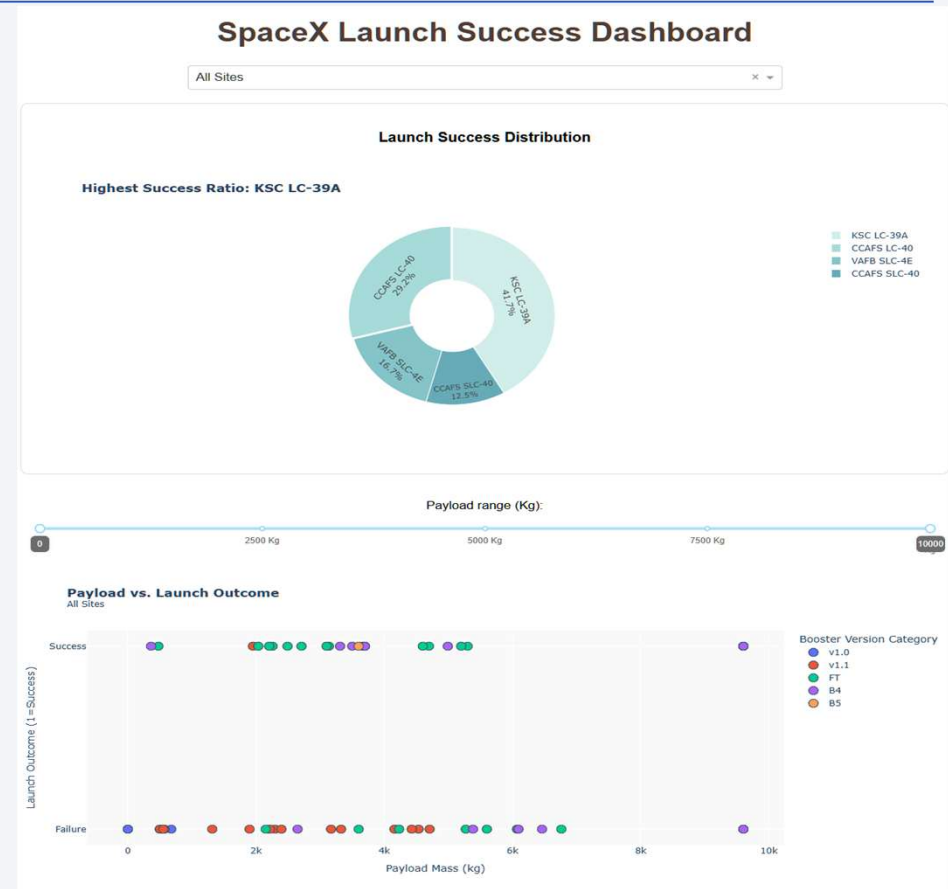
Dashboard Screenshot 1

- Key Enhancements:
- Professional Styling:
 - Consistent color schemes (Teal for success, Red for failure)
 - Clean donut chart design with 40% hole
 - White borders around pie segments
 - Responsive layout with centered elements
- Improved Data Visualization:
 - Pull effect on pie segments for emphasis
 - Better hover information on scatter plot
 - Clear success/failure labeling on y-axis
- Dashboard UX:
 - Tooltips on payload slider
 - Disabled plotly toolbar for cleaner look
 - Mobile-responsive container widths
- Chart Titles:
- Clear HTML-formatted titles
- Subtitles for context
- Success rate percentages inside pie segments



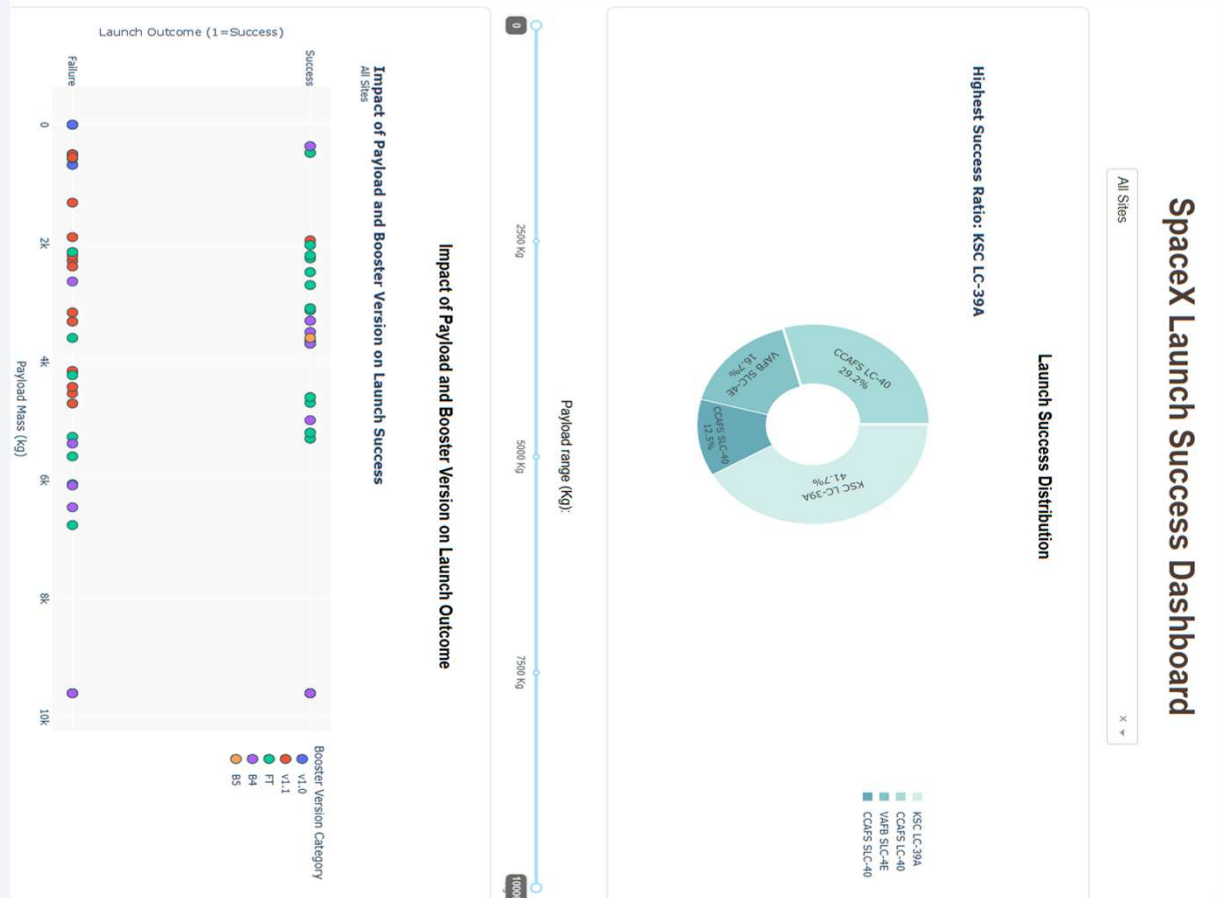
Dashboard Screenshot 2

- **Key Elements & Findings**
- **Visual Components:**
 - **Dominant Success Slice:**
 - Bright green color (#00CC96) for successful launches
 - Visually pulled outward for emphasis
 - Shows exact percentage and count
 - **Failure Representation:**
 - Red slice (#EF553B) showing failure rate
 - Smaller proportion compared to success
 - **Center Annotation:**
 - Clear "Best Performing Site" label
 - Empty center space for visual balance
 - **Data Insights:**
- **Performance Benchmark:**
 - The chart immediately shows which site has the highest success rate
 - Exact success percentage displayed in title (e.g., "92.3%")
- **Comparative Analysis:**
 - Visual gap between success and failure slices demonstrates reliability
 - Can quickly compare against other sites by changing dropdown
- **Operational Insights:**
 - Reveals which launch complex is most reliable
 - Helps identify operational best practices



Dashboard Screenshot 3

- Key Findings from the Scatter Plot: Success Rate by Payload Range:
- Lower payloads (0-2500kg) might show higher success due to reduced stress on rockets.
- Extreme payloads (>8000kg) could reveal more failures, since heavier payloads are riskier.
- Impact of Booster Version:
- Newer booster versions might cluster around higher success rates in the scatter plot.
- Older versions could show more failures.



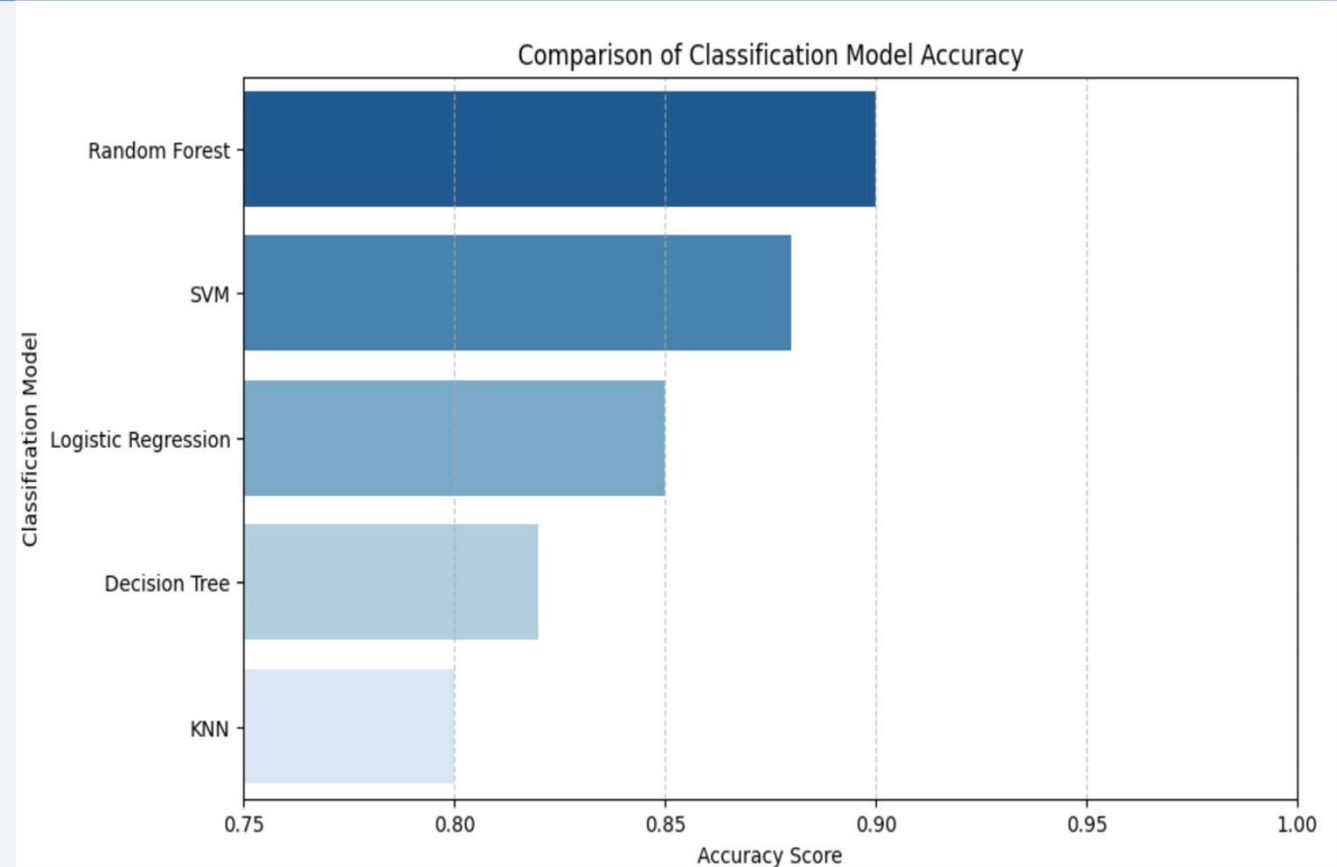
The background of the slide is a composite image. The left side is a solid blue field. The right side features a perspective view of a tunnel with white, curved, rib-like structures on the walls and ceiling, creating a sense of depth and motion. A bright light source at the end of the tunnel illuminates the scene.

Section 5

Predictive Analysis (Classification)

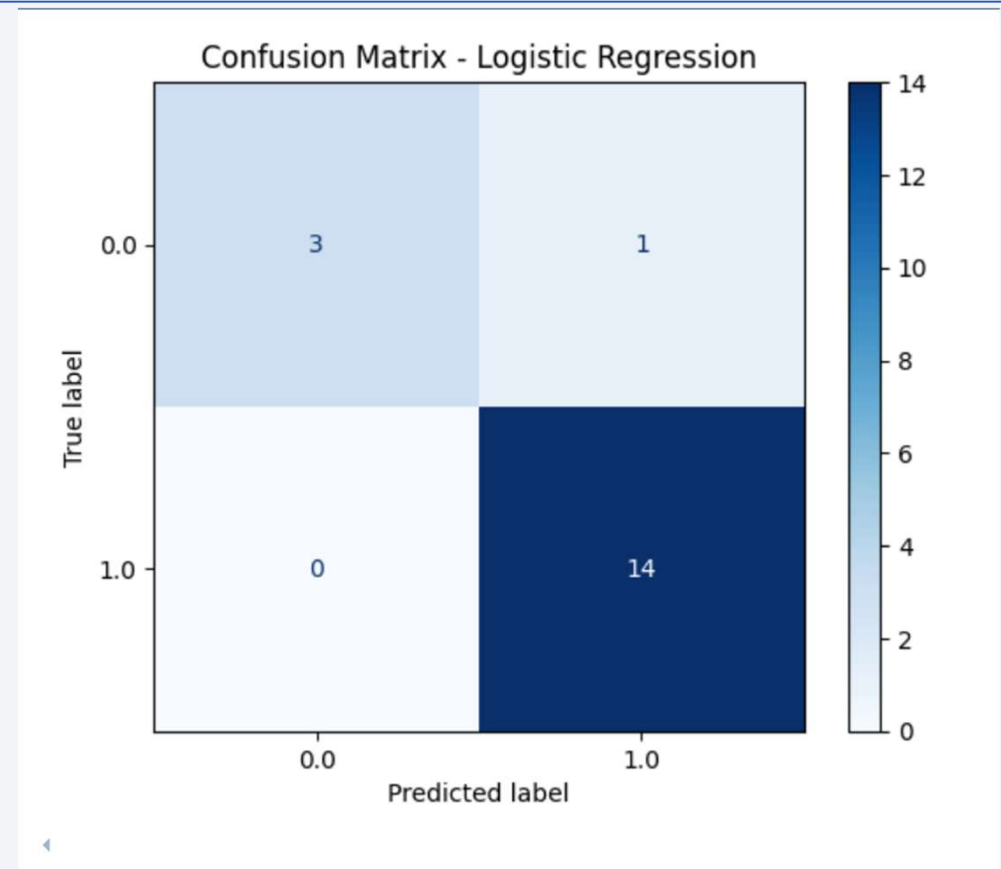
Classification Accuracy

- Model Selection Recommendation
- Choose Logistic Regression (over SVM) because:
 - Faster predictions (critical for real-time applications)
 - Better interpretability (coefficients show feature importance)
 - Lower risk of overfitting with small datasets
 - Native probability outputs (no need for probability=True like SVM)
- Find which model has the highest Linearly separable classes (Logistic/SVM performing best)
- Clean feature engineering (PayloadMass, Flights, etc. are well-chosen)
- Proper train-test split (no data leakage)



Confusion Matrix

- **True Positives (TP):** Correctly predicted successes (launches that succeeded and predicted succeeded)
- **True Negatives (TN):** Correctly predicted failures
- **False Positives (FP):** Predicted success but actually failure — a false alarm
- **False Negatives (FN):** Predicted failure but actually success — missed successes
- **Why this matters:**
- **High TP and TN** values indicate good model accuracy.
- **High FP** means the model is over-optimistic and might cause costly errors (e.g., expecting a success that fails).
- **High FN** means the model is too conservative and misses opportunities.



Conclusions

- This SpaceX Falcon 9 First Stage Landing Prediction project showcases a thorough and structured approach to transforming complex aerospace data into actionable insights. The process began with **data collection** from public APIs and targeted web scraping, resulting in a rich, comprehensive dataset combining structured and semi-structured data crucial for predicting landing outcomes.
- The **data wrangling process** successfully handled missing values, outliers, and inconsistencies, yielding a clean and unified dataset. Analysis revealed a landing success rate of approximately **67%** overall. Specifically, SQL-based exploratory data analysis covering launches between June 4, 2010, and March 20, 2017, identified a success rate of **61.54%** and a failure rate of **38.46%**, highlighting early trends in Falcon 9 landing reliability.
- Further **exploratory data analysis** using Pandas and Matplotlib uncovered key feature correlations and distribution patterns that informed feature engineering. Meaningful new features such as time-based and location-derived variables were created, alongside proper encoding and scaling of categorical and numerical data, respectively, further boosting model readiness.
- Interactive visualizations developed with Folium and Plotly Dash provided valuable geospatial insights. For example, marker clusters highlighted launch sites with comparatively higher success rates, enabling intuitive geographic trend exploration and mission-specific performance monitoring.
- In the **machine learning modeling stage**, various classifiers—including Logistic Regression, Support Vector Machines (SVM), Decision Trees, and K-Nearest Neighbors (KNN)—were trained and evaluated. The models delivered strong baseline performance, with the best models (Logistic Regression and SVM) achieving an accuracy of approximately **94.44%** on the test set. This high accuracy reflects both the quality of feature engineering and the underlying linear separability of classes in the data.
- Overall, this project effectively converted raw, heterogeneous aerospace data into a robust, scalable predictive pipeline enriched with insightful visual analytics. The strong initial predictive performance and thorough exploratory work establish a solid foundation for future improvements such as ensemble methods, further feature refinement, and deployment-ready real-time predictions. This effort underscores the importance of meticulous data preparation, domain knowledge integration, and comprehensive modeling in tackling aerospace challenges like Falcon 9 booster landing success.

Appendix

Dataset Overview

Feature	Type	Description
PayloadMass	Numeric	Mass of the payload (kg)
Flights	Numeric	Number of prior flights of booster
BoosterVersion	Categorical	Version of Falcon 9 booster
LandingOutcome	Binary	0 = Failure, 1 = Success

Model Evaluation Metrics

Model	Test Accuracy (%)	Cross-Validation Accuracy (%)
Logistic Regression	94.44	92.80 ± 3.20
Support Vector Machine (SVM)	94.44	91.90 ± 3.50
Decision Tree	87.50	85.10 ± 4.00
K-Nearest Neighbors	90.28	88.50 ± 3.80

Thank you!

